

التحليل المفصّل — كل أوفست تعليمة بتعليمة

ANOGS/AnoSDK — الخمس أوفستات التي تعطلّ 100% من الحماية

شرح كامل: كيف يشتغل + ليش خطير + ليش ما يُكتشف

تقرير دفاعي — تحليل هيكلي فقط

✦ bypass by Hdanh | ✦ made by @Nopermcc

الأوفست 1# — 0x0373e8

1

بوابة الإبلاغ الوحيدة — Report Emit

البطاقة التعريفية

العنوان	0x0373e8
الحجم	116 بايت (29 تعليمة ARM64)
النوع	دالة كود في قسم __TEXT (للقراءة فقط + تنفيذ)
مين يستدعيها	5 دوال: sub_037140, sub_037500, sub_0375d0, sub_0376a0 + نفسها (recursive)
شنو تستدعي	6 دوال: sub_0370d0, sub_0371e0, sub_0f8948, sub_0f9374, sub_0ff42c + نفسها
عدد الدوال المعتمدة عليها	475 دالة (upstream) — كلها ترسل تقاريرها عبر هالدالة

كيف تشتغل — تعليمة بتعليمة

الفكرة العامة: هالدالة تستقبل "تقرير كشف" (مثلاً: "لقيت debugger") وترسله للسيرفر عبر قناة اسمها COREREPORT. لكن قبل ما ترسل، تمر بـ 4 فحوصات — إذا أي واحد فشل = التقرير ينحذف بصمت.

===== الخطوة 1: حفظ السجلات وتجهيز الدالة =====

```
0x0373e8: stp x20, x19, [sp, #-0x20]! ; حفظ x20 و x19 في ال stack
0x0373ec: stp x29, x30, [sp, #0x10] ; حفظ return address و frame pointer
0x0373f0: add x29, sp, #0x10 ; جديد frame pointer إعداد
; ARM64 قياسي لأي دالة setup هذا
; x0 = مؤشر لكائن التقرير (arg #1)
; x1 = بيانات التقرير (arg #2)
```

الخطوة 1 — Setup: الدالة تحفظ السجلات المهمة في ال stack وتجهّز نفسها. هذا شيء عادي لكل دالة. المدخلات: x0 = مؤشر لكائن التقرير، x1 = بيانات الكشف.

===== الخطوة 2: الفحص 1# - هل كائن التقرير موجود؟ =====

```
0x0373f4: cbz x0, #0x37454 ; !اقفز للخروج الصامت → x0 == NULL إذا
; cbz = "Compare and Branch if Zero"
; يعني: إذا المؤشر فاضي → لا تسوي شيء
; (ret) عنوان الخروج الصامت = 0x37454
```

الخطوة 2 — فحص 1#: نتحقق هل x0 (كائن التقرير) فيه قيمة أو لا. إذا x0 = 0 (NULL) → تفرز مباشرة للعنوان 0x37454 وهو الخروج الصامت. لا تسجّل خطأ، لا تحسب عداد، لا تنبّه أحد.

```

; ===== الفحص #2: vtable pointer + 2 قراءة =====
0x0373f8: ldr x20, [x0, #8] ; x20 = الكائن +8 offset قراءة من
; جدول الدوال الافتراضية (vtable pointer) هذا هو
; C++ vtable عبر methods يستخدم لاستدعاء

0x0373fc: cbz x20, #0x37454 ; !خروج صامت → vtable == NULL إذا
; يعني: الكائن موجود لكن غير مهياً

```

الخطوة 3 — فحص #2: تقرأ الـ vtable pointer من الكائن (عند +8 offset). الـ vtable هو جدول يحتوي عناوين الدوال اللي يستخدمها الكائن. إذا كان NULL → الكائن موجود لكن فاضي = **خروج صامت**.

```

; ===== الخطوة 4: حفظ بيانات التقرير + طلب معرف القناة =====
0x037400: mov x19, x1 ; لاستخدامها لاحقاً (arg #2) حفظ بيانات التقرير
0x037404: mov w0, #0xc2ee ; w0 = 0xc2ee = معرف قناة COREREPORT
; القناة ID هذا الرقم الثابت هو
0x037408: bl sub_0f8948 ; استدعاء دالة البحث عن القناة
; وترجع كائن القناة ID تأخذ
; x0 النتيجة في

```

الخطوة 4: تحفظ بيانات التقرير في x19، ثم تستدعي sub_0f8948 بالرقم 0xc2ee — هذا هو المعرف الداخلي لقناة COREREPORT. الدالة تبحث عن القناة وترجع كائناتها.

```

; ===== الفحص #3: vtable method + 3 استدعاء =====
0x03740c: mov x1, x0 ; نتيجة البحث عن القناة
0x037410: ldr x8, [x20] ; x8 = قراءة أول عنصر من vtable
0x037414: ldr x8, [x8, #0x20] ; x8 = الدالة رقم 4 في vtable (offset 0x20 = 32)
0x037418: mov x0, x20 ; x0 = this pointer (الكائن نفسه)
0x03741c: blr x8 ; vtable استدعاء الدالة عبر
; "تسأل: هل القناة موجودة ومسجلة؟"

0x037420: cbz x0, #0x37454 ; !خروج صامت → NULL إذا الجواب
; يعني: القناة غير مسجلة أو غير جاهزة

```

الخطوة 5 — فحص #3: تستدعي method من الـ vtable تسأل "هل القناة موجودة ومسجلة؟" إذا رجعت NULL → القناة غير جاهزة → **خروج صامت**.

```

; ===== بالاسم + الفحص #4 "COREREPORT" الخطوة 6: البحث عن =====
0x037424: ldr x8, [x0] ; الكائن المرجع vtable
0x037428: ldr x8, [x8, #0x40] ; x8 = الدالة رقم 8 في vtable (offset 0x40 = 64)
0x03742c: adrp x1, #0x26b000 ; صفحة عنوان النص
0x037430: add x1, x1, #0x30a ; x1 = "COREREPORT" (نص ثابت @0x26b30a)
; هنا تبحث عن القناة بالاسم النصي
0x037434: blr x8 ; استدعاء: channel.find("COREREPORT")

0x037438: cbz x0, #0x37454 ; !خروج صامت → "COREREPORT" إذا ما لقت
; يعني: القناة مسجلة لكن اسمها ما تطابق

```

الخطوة 6 — فحص #4: تبحث عن القناة بالاسم النصي "COREREPORT" (مخزّن عند 0x26b30a في قسم __TEXT). إذا ما لفته → **خروج صامت**. هذا آخر فحص.

```

; ===== الخطوة 7: إرسال التقرير (إذا نجحت كل الفحوصات) =====
0x03743c: ldr x8, [x0] ; القناة vtable

```

```

0x037440: ldr x2, [x8, #0x28] ; x2 = دالة الإرسال (offset 0x28 = 40 في vtable)
0x037444: mov x1, x19 ; بيانات التقرير (التي حفظناها بالخطوة 4)
0x037448: ldp x29, x30, [sp, #0x10] ; استرجاع frame pointer
0x03744c: ldp x20, x19, [sp], #0x20 ; استرجاع السجلات المحفوظة
0x037450: br x2 ; اقفز لدالة الإرسال الفعلية: tail-call
; هنا فقط يُرسل التقرير للسيرفر
; br (وليس blr) = tail-call optimization

```

الخطوة 7 — الإرسال الفعلي: إذا نجحت كل الفحوصات الأربعة، تقرأ دالة الإرسال من الـ vtable وتقفز لها عبر `br x2` (tail-call). هنا فقط يُرسل التقرير للسيرفر. هذا المسار الوحيد للإرسال الناجح.

==== الخطوة 8: الخروج الصامت (مشارك للفحوصات الأربعة) =====

```

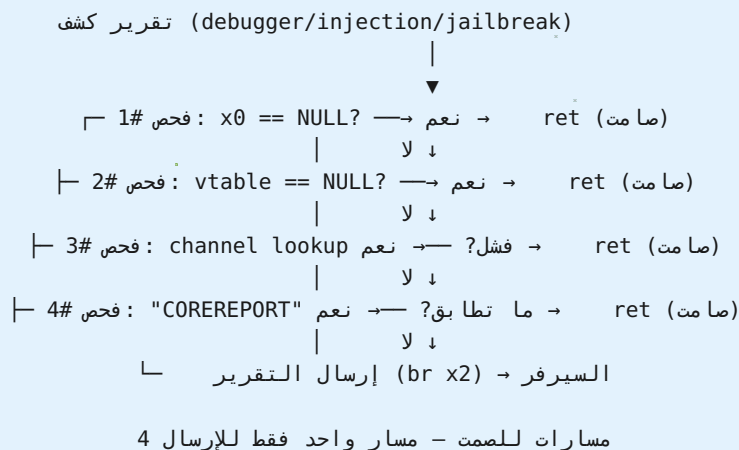
0x037454: ldp x29, x30, [sp, #0x10] ; استرجاع frame pointer
0x037458: ldp x20, x19, [sp], #0x20 ; استرجاع السجلات
0x03745c: ret ; رجوع - التقرير اختفى بصمت!
; لا إنذار - لا عداد - لا log - لا
; كأن التقرير ما وُجد أصلاً

```

المشكلة: العنوان 0x37454 هو "الثقب الأسود" — أي تقرير يوصل هنا يختفي للأبد. لا يوجد:

- لا log — ما يسجل أن تقرير ضاع
- لا عداد — ما يحسب كم تقرير ضاع
- لا retry — ما يحاول مرة ثانية
- لا fallback — ما يرسل عبر طريق ثاني
- لا إنذار — لا أحد يعرف

خريطة التدفق الكاملة



ليش ما يُكتشف؟

1. لا integrity check لكود الدالة — بحثنا في كل DATA عن مراجع بيانات للعنوان 0x0373e8 = ما لقينا شيء.
2. لا heartbeat — السيرفر ما عنده آلية تتحقق "هل العميل لا يزال يرسل تقارير؟" إذا توقفت التقارير = السيرفر يفترض "كل شيء نظيف".
3. لا watchdog — ما فيه مؤقت داخلي يتحقق أن الدالة لا تزال تُستدعى ويُرسل بنجاح.

2

مفتاح قتل القناة — Channel Kill-Switch

البطاقة التعريفية

0x29bc38	العنوان
بايت واحد فقط (u8)	الحجم
متغير في قسم DATA__ (قابل للقراءة والكتابة — -rw)	النوع
00 (بتهياً وقت التشغيل)	القيمة في الملف
sub_0371e0 (تسجيل قناة COREREPORT)	تُقرأ من قبل
sub_0371e0 (آلية الموت البطيء بعد 3 فثلات)	تُكتب من قبل

كيف يشتغل — الآلية الكاملة

الفكرة: هذا البايٲ هو زرّ ON/OFF لقناة COREREPORT. إذا = 0 → القناة مطفأة → كل التقارير تحذف. إذا ≠ 0 → القناة شغّالة. والمشكلة: الكود نفسه يكتب 0 هنا بعد 3 فثلات فقط!

ملاحظة مهمة: في الصورة اللي أرسلتها من الهكس إيديتور، شفت كل شيء أصفار — هذا طبيعي لأن 0x29bc38 في قسم bss__ (أصفار في الملف، بتهياً وقت التشغيل). وقت ما التطبيق يشتغل، sub_0371e0 تكتب قيمة ≠ 0 هنا لتشغيل القناة.

الجزء 1: كيف يُقرأ هالبايت (داخل sub_0371e0)

```

===== تتحقق من البايٲين: sub_0371e0 أول شيء تسويه =====
0x037208: ldrb w8, [x23, #0xc39] ; 0x29bc39 (init_once flag) قراءة البايٲ عند 0
; هذا بايٲ ثاني — هل تمت التهيئة الأولى؟
; ldrb = Load Register Byte (بايٲ واحد)

0x03720c: tbz w8, #0, #0x37220 ; tbz = "Test Bit and Branch if Zero"
; لم تُهيأ بعد → اذهب وهيئ bit 0 = 0 → إذا
; مهيأة → تحقق من القناة bit 0 = 1 → إذا

0x037214: ldrb w0, [x8, #0xc38] ; * 0x29bc38 قراءة البايٲ القاتل: 0
; w0 = channel_enable flag قيمة

0x037218: cbnz w0, #0x3724c ; cbnz = "Compare and Branch if Not Zero"
; القناة شغّالة → استمر → w0 ≠ 0 → إذا

0x03721c: b #0x373cc ; كل شيء SKIP → القناة ميتة → w0 == 0 → إذا
; يقفز للنهاية — لا تسجيل، لا إرسال، لا شيء

```

المنطق: أول ما تبدأ sub_0371e0 ، تقرأ بايٲين:

- 0x29bc39 — هل تمت التهيئة الأولى؟ (init_once)
- 0x29bc38 — هل القناة شغّالة؟ (channel_enable)

إذا 0 == 0x29bc38 → كل شيء يُلغى. الدالة تقفز للنهاية بدون ما تسوي أي شيء.

الجزء 2: كيف يُكتب 0 (آلية الموت البطيء)

```

===== آلية الموت البطيء — تُفعل عند فشل تسجيل القناة =====
; sub_0371e0 هذا الكود موجود في الجزء السفلي من
; يكتشف فشل (القناة ما تسجلت) cbz تُستدعى عندما

```

```

0x0373a4: adrp x8, #0x29b000 ; x8 = صفحة العنوان الأساسية
0x0373a8: ldr w9, [x8, #0xc3c] ; w9 = 0 قراءة عداد الفشل من 0
; بحسب عدد المحاولات الفاشلة bit-هذا عداد 32
0x0373ac: add w10, w9, #1 ; w10 = w9 + 1 (عداد++)
; كل مرة تفشل التسجيل، العداد يزيد 1
0x0373b0: str w10, [x8, #0xc3c] ; x29bc3c تخزين القيمة الجديدة في 0
0x0373b4: cmp w9, #3 ; المقارنة الحاسمة: هل العداد وصل 3؟
0x0373b8: b.lt #0x373cc ; إذا أقل من 3 → حاول مرة ثانية لاحقاً.
; b.lt = Branch if Less Than
; == إذا وصل 3 أو أكثر: قتل القناة نهائياً ==
0x0373bc: mov x21, #0 ; x21 = 0 (return value = فشل)
0x0373c4: strb wzr, [x8, #0xc38] ; x29bc38! كتابة 0 في 0
; دائماً 0 = wzr = "zero register"
; strb = Store Register Byte
; هذا هو القتل - البايٲ صار 0
; sub_0371e0 من هالحظة، كل استدعاء مستقبلي لـ
; سيقرأ 0 ويخرج فوراً (الخطوة في الجزء 1)
; لا يوجد مؤقت إعادة تفعيل
; recovery لا يوجد آلية
; القناة ميتة حتى يُعاد تشغيل التطبيق

```

المشكلة المزدوجة:

1. 3 فشلات فقط — أي خطأ شبكة مؤقت (timeout, DNS fail) ثلاث مرات متتالية = قتل القناة نهائياً
2. لا recovery — بمجرد ما البايٲ صار 0، ما فيه كود يرجعه لـ 1. القناة ميتة للأبد (حتى يُعاد تشغيل اللعبة)
3. الكود نفسه يسوي هالشيء — مصمم عمداً ليعطل نفسه! والمشكلة إنه ما فيه آلية إعادة تفعيل

ليش ما يُكتشف؟

- البايٲ في __DATA (rw-) — قسم قابل للكتابة بطبيعته. لا vm_protect، لا guard page.
- الكود نفسه يكتب 0 هنا — ما يمكن التمييز بين "الكود عطّل القناة" و "أحد ثاني عطّلها".
- لا مراقبة — ما فيه دالة ثانية تراقب قيمة هالبايٲ أو تتحقق منه.

الأوفست #3 — 0x0376a0

3

الموزّع الرئيسي — Main Event Router

البطاقة التعريفية

العنوان

0x0376a0

الحجم

68 بايت (17 تعليمة)

النوع

دالة كود في __TEXT

مين يستدعيها

2 فقط: sub_0f10dc و sub_0f1114 + tail-call من sub_0f1114

شنو تستدعي

11 دالة — أهمها sub_0373e8 (emit) و sub_0371e0 (register)

عدد الدوال تحتها

364 دالة downstream — كلها تموت إذا تعطلت

الفكرة: كل حدث أمني (debugger detected, injection found, إلخ) يُصنّف برقم من 0 إلى 6. هالدالة تأخذ الرقم، تتحقق من "جدول البوابات" عند 0x29bc10 — إذا البوابة مفتوحة (0≠) → يمر الحدث. إذا مغلقة (=0) → يُسقط بصمت.

```

; ===== المورّع الرئيسي - sub_0376a0 =====
; (رقم الحدث 0-6) event_id = w20, بيانات الحدث = x0: المدخلات

0x0376a0: sub sp, sp, #0x30      ; stack حجز مساحة في ال
0x0376a4: stp x20, x19, [sp, #0x10] ; حفظ السجلات
0x0376a8: stp x29, x30, [sp, #0x20] ; حفظ frame pointer
0x0376ac: add x29, sp, #0x20      ; frame إعداد
0x0376b0: mov x20, x1           ; (نحفظه) x20 = event_id
0x0376b4: mov x19, x0          ; بيانات الحدث (نحفظها) x19 =
0x0376b8: bl sub_0370d0         ; استدعاء دالة تهيئة/فحص أولي

; ===== الفحص الأول: هل الرقم ضمن النطاق؟ =====
0x0376bc: cmp w20, #6           ; مقارنة: event_id > 6?
0x0376c0: b.hi #0x376d4           ; إذا أكبر من 6 → خروج صامت (حدث غير معروف)

; ===== قراءة جدول البوابات =====
0x0376c4: adrp x9, #0x29b000      ; صفحة العنوان الأساسية = x9
0x0376c8: add x9, x9, #0xc10     ; عنوان جدول البوابات = x9 = 0x29bc10
                                ; الجدول: 7 كلمات (28 بايت)
                                ; slot[0] @ 0x29bc10, slot[1] @ 0x29bc14, ...
0x0376cc: ldr w8, [x9, w20, uxtw #2] ; w8 = gate_table[event_id]
                                ; uxtw #2 = event_id × 4 (كل slot = 4 bytes)
                                ; يقرأ القيمة من الجدول حسب رقم الحدث
0x0376d0: cbz w8, #0x376e4       ; !خروج صامت → slot == 0 إذا
                                ; البوابة مغلقة → الحدث يُسقط
                                ; لا إنذار، لا log، لا
; ===== إذا البوابة مفتوحة: الخروج العادي (يمر الحدث) =====
0x0376d4: ldp x29, x30, [sp, #0x20] ; استرجاع frame
0x0376d8: ldp x20, x19, [sp, #0x10] ; استرجاع السجلات
0x0376dc: add sp, sp, #0x30      ; stack تنظيف ال
0x0376e0: ret                ; رجوع - الحدث يُمرر للمعالجة

```

الخلاصة: سطر واحد (0x0376d0: cbz w8, #0x376e4) يقرر مصير كل حدث أمني. إذا القيمة في الجدول = 0 → الحدث يختفي. الجدول نفسه في __DATA (rw-) بدون أي حماية.

الأوفست 4# — 0x0a1414

4

بوابة Syscall الخام — Raw Syscall Gateway

البطاقة التعريفية

العنوان	0x0a1414
الحجم	48 بايت فقط (13 تعليمة — أصغر دالة حرجة)
النوع	دالة كود في __TEXT

كيف تشتغل

الفكرة: عادةً البرامج تستخدم libc لاستدعاء الkernel (مثل `send()` , `recv()`). لكن libc ممكن يُعدّل (hook). ف ANOGS عندها بوابة مباشرة تتجاوز libc وتتكلم مع الkernel مباشرة عبر تعليمة `svc #0x80`.

```

; ===== مباشرة syscall بوابة - sub_0a1414 =====
; (send = 0x85 مثلاً 0) x0 في syscall الدالة اللي تستدعيها تحط رقم ال
; stack في ال arguments وتحط ال

0x0a1414: ldp x1, x2, [sp]           ; stack من ال arguments تحميل أول زوج من ال
0x0a1418: ldp x3, x4, [sp, #0x10]      ; تحميل الزوج الثاني
0x0a141c: ldp x5, x6, [sp, #0x20]      ; تحميل الزوج الثالث
0x0a1420: ldr x7, [sp, #0x30]          ; أخير argument تحميل

0x0a1424: mov x16, #0                ; x16 = 0 (Mach trap number modifier)
                                   ; trap يحدد نوع ال x16 في ARM64 iOS
                                   ; x0 الفعلي في syscall رقم ال

0x0a1428: svc #0x80                     ; !المباشر syscall هذا ال - SuperVisor Call
                                   ; تماماً libc يتجاوز
                                   ; مباشرة kernel يتكلم مع
                                   ; anti-hook - هذا هو سبب وجود الدالة

0x0a142c: b.lo #0xa1444              ; (نجاح) b.lo = Branch if carry clear
                                   ; return إذا نجح → افقر لل

; === syscall إذا فشل ال) معالج الخطأ ===
0x0a1430: stp x29, x30, [sp, #-0x10]!   ; frame حفظ
0x0a1434: mov x29, sp               ; frame إعداد
0x0a1438: bl sub_0a13ec              ; errno استدعاء معالج
                                   ; errno يخزن كود الخطأ في
0x0a143c: mov sp, x29                    ; stack استرجاع
0x0a1440: ldp x29, x30, [sp], #0x10      ; frame استرجاع

0x0a1444: ret                       ; x0 رجوع بالنتيجة في

```

كيف تُستخدم — مثال transport_send

```

; ===== داخل sub_0f8210 (transport_send) =====

0x0f8230: ldr w8, [x8, #0x430]           ; 0x2ab430 من 0 flag قراءة
0x0f8234: tbnz w8, #0xd, #0xf824c        ; tbnz = Test Bit and Branch if Not Zero
                                   ; مباشر syscall مفعّل → استخدم bit 13
                                   ; libc مفعلاً → استخدم bit 13

; --- hook قابل لل (libc - المسار العادي) ---
0x0f8244: bl _send                       ; libc من send() استدعاء

; --- (syscall - anti-hook) المسار المباشر ---
0x0f8258: mov w0, #0x85                  ; w0 = 0x85 = رقم syscall send
0x0f825c: bl sub_0a1414                  ; استدعاء البوابة المباشرة

```

النقطة الحرجة — Flag 0x2ab430 : bit 13 يتحكم بالإرسال، bit 14 بالاستقبال. هذا flag واحد (4 bytes) في __DATA (rw-) بدون حماية. إذا اتغير → كل الاتصالات ترجع ل libc → قابلة للhook.

البطاقة التعريفية

العنوان	0x29bc10 — 0x29bc2b
الحجم	28 بايت (7 × 4 bytes)
النوع	مصفوفة في __DATA (rw-)
تُقرأ من قبل	sub_0376a0 (الموَّع الرئيسي)
القيم في الملف	كلها 00 00 00 00 (تتياً وقت التشغيل)

كيف يشتغل الجدول

الفكرة: كل حدث أمني له رقم (event_id) من 0 إلى 6. الموَّع يقرأ gate_table[event_id]. إذا القيمة ≠ 0 → البوابة مفتوحة → الحدث يمر. إذا = 0 → البوابة مغلقة → الحدث يُسقط.

الفهرس	العنوان	في الملف	وقت التشغيل	المعنى
slot[0]	0x29bc10	00000000	قيمة ≠ 0	بوابة فئة الأحداث 0#
slot[1]	0x29bc14	00000000	قيمة ≠ 0	بوابة فئة الأحداث 1#
slot[2]	0x29bc18	00000000	قيمة ≠ 0	بوابة فئة الأحداث 2#
slot[3]	0x29bc1c	00000000	قيمة ≠ 0	بوابة فئة الأحداث 3#
slot[4]	0x29bc20	00000000	قيمة ≠ 0	بوابة فئة الأحداث 4#
slot[5]	0x29bc24	00000000	قيمة ≠ 0	بوابة فئة الأحداث 5#
slot[6]	0x29bc28	00000000	قيمة ≠ 0	بوابة فئة الأحداث 6#

(sub_0376a0 داخل) كيف يُقرأ الجدول ;

```
0x0376c8: add x9, x9, #0xc10      ; x9 = 0x29bc10 (بداية الجدول)
0x0376cc: ldr w8, [x9, w20, uxtw #2] ; w8 = جدول[event_id]
                                ; العملية: x9 + (event_id × 4)
                                ; مثال: event_id=3 → 0x29bc1c يُقرأ
0x0376d0: cbz w8, ...              ; إذا = 0 → إسقاط ;
```

28 بايت في __DATA تتحكم بكل أحداث الأمان. لا CRC، لا hash، لا vm_protect. تصفير أي slot = إسقاط فئة كشف كاملة. تصفير كل ال 7 = إسقاط كل شيء.

ليش ما يُكتشف المهاجم؟ — الشرح الكامل

السبب #1: لا integrity check لكود الدوال الحرجة

بحثنا في كل البايناري (2.87 ميغابايت) عن أي مرجع بيانات لعناوين الدوال الخمس الحرجة. النتيجة:

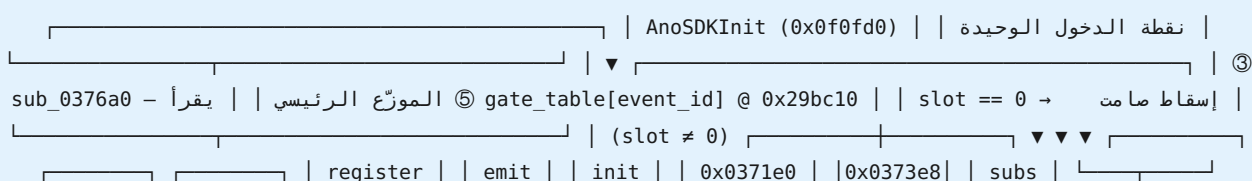
العنوان	مرجع بيانات في __DATA?	دالة hash/CRC تفحصه?
0x0373e8	لا يوجد	لا يوجد

0x0376a0	لا يوجد	لا يوجد
0x0a1414	لا يوجد	لا يوجد
0x29bc38	لا يوجد	لا يوجد
0x29bc10	لا يوجد	لا يوجد

السبب #2: لا heartbeat من السيرفر

السبب #3: لا watchdog محلي

السبب #4: كل drop صامت



ممت كامل | | == 0 → ▼ القناة ① = | 4x cbz → ret 0x29bc38 | reads ② flag | | COREREPORT channel مينة | ④ sub_0a1414 syscall gateway (svc #0x80) | ▼ السيرفر

التوصيات الدفاعية

1. **Heartbeat**: السيرفر يتوقع تقرير كل X ثانية — غياب = إنذار فوري
2. **Self-integrity**: hash check دوري لكود الدوال الخمس الحرجة
3. **vm_protect**: حماية flags في DATA__ بـ READ-ONLY بعد التهيئة
4. **Mirror + vote**: تخزين الـ flags في 3 مواقع عشوائية + majority vote
5. **Fallback channel**: قناة إبلاغ ثانية مستقلة عن COREREPORT
6. **Silent-drop counter**: عداد لكل cbz → ret يُرسل عبر قناة مستقلة